SOFTWARE ENGINEERING

Complete Interview & Placement Notes

Enhanced Edition — with Diagrams & Placement Roadmap

SDLC • Design Patterns • Architecture • Agile • Testing • UML • On/Off-Campus Strategy

Table of Contents

1. Software Development Life Cycle (SDLC)
2. Requirements Engineering
3. Software Design Principles
4. UML — Unified Modeling Language
5. Agile Methodology & Scrum
6. Software Testing
7. Software Metrics & Project Management
8. SCM, Version Control & DevOps
9. Software Maintenance, Reliability & Ethics
10. Top Interview Questions & Quick Answers
11. PLACEMENT ROADMAP — On-Campus vs Off-Campus (New)
12. Quick Revision Cheat Sheet

This edition keeps every topic from the original notes and adds: deeper explanations under each heading, an original diagram for every major model or framework, and a brand-new placement-strategy section covering both on-campus and off-campus hiring so you know exactly what to expect and how to prepare for each.

1. Software Development Life Cycle (SDLC)

SDLC is the structured process software teams follow to plan, build, test, and deliver a product. Interviewers ask about it because it shows whether you understand how real projects are organized, not just how to write code. Think of it as a recipe: skip a step (like requirements gathering) and the final product usually disappoints the customer, no matter how well the code was written.

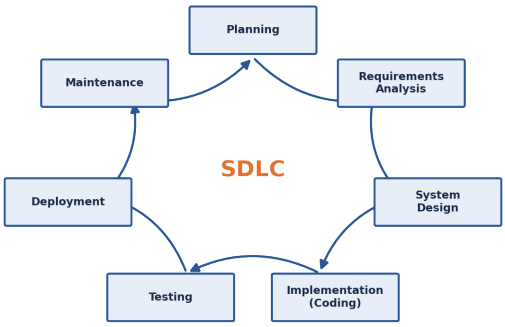

Figure 1.1: The seven SDLC phases, shown as a repeating cycle.

SDLC Phases

Phase	Activities / Output
Planning	Feasibility study, resource estimation, project scheduling
Requirements	SRS (Software Requirements Specification) document
System Design	HLD (High-Level Design), LLD (Low-Level Design)
Implementation	Coding, unit testing by developers
Testing	Integration, system, acceptance testing
Deployment	Release to production environment
Maintenance	Bug fixes, enhancements, performance tuning

SDLC Models — Explained

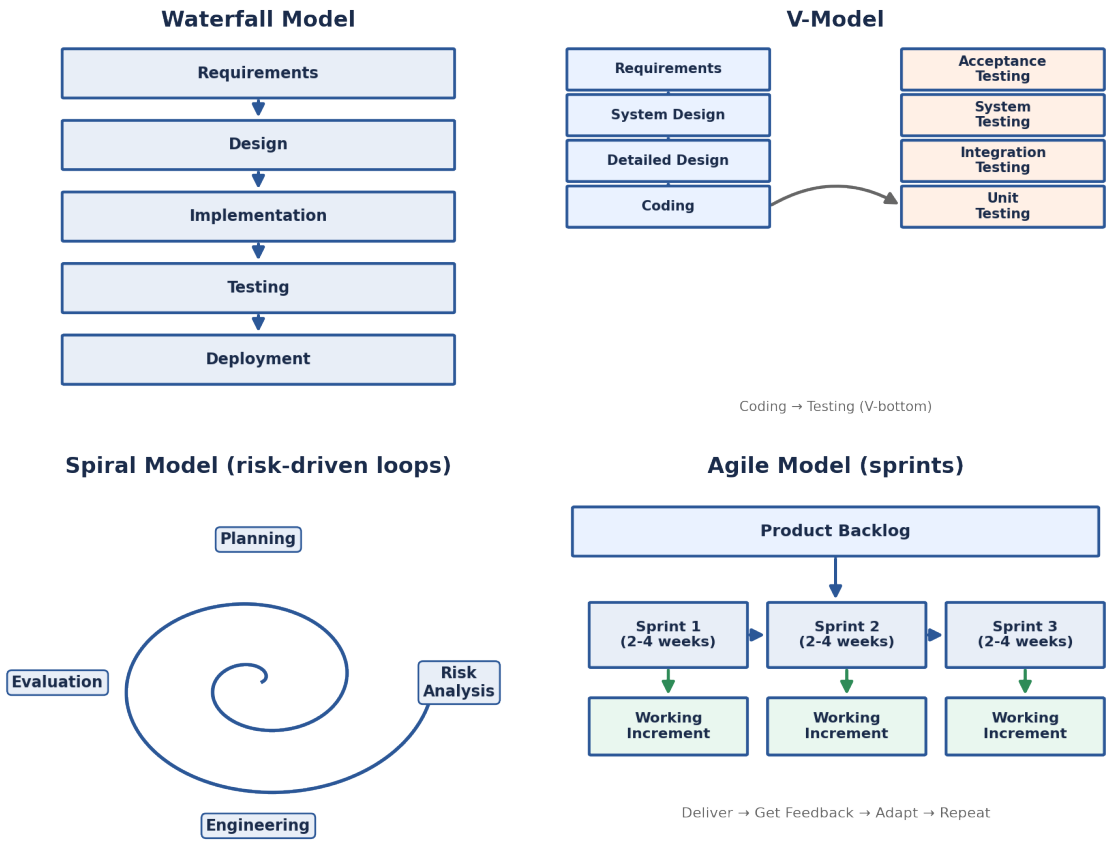

Figure 1.2: Waterfall, V-Model, Spiral, and Agile compared side by side.

Waterfall Model

A strictly sequential approach: each phase (requirements, design, implementation, testing, deployment) must finish before the next begins, like water flowing downhill and never back up. It works well when requirements are frozen and well understood — for example, government or compliance projects. Its biggest weakness is that no working software exists until very late, so mistakes in early requirements are expensive to fix.

V-Model (Verification & Validation)

An extension of Waterfall where every development phase has a matching test phase planned in parallel, so testing isn't an afterthought. It's the go-to choice for safety-critical systems (medical devices, aerospace, banking cores) where a bug discovered after release could be catastrophic.

Iterative Model

Instead of one long pass through the SDLC, the team repeats short cycles, each producing a partial but working version of the product. Requirements are allowed to evolve as the team learns more with each iteration — useful when the full scope isn't clear up front.

Spiral Model

Combines Waterfall's structure with prototyping and adds an explicit risk-analysis step at every loop: Planning → Risk Analysis → Engineering → Evaluation. Each loop around the spiral tackles the riskiest unknowns first, which is why it suits large, expensive, high-risk projects where a wrong turn is very costly.

Agile Model

Development happens in short sprints (typically 2–4 weeks), each ending with a potentially shippable increment. Agile favours customer collaboration over rigid contracts and responding to change over following a fixed plan. Scrum, Kanban, XP, and SAFe are all frameworks that implement Agile values in different ways.

RAD (Rapid Application Development)

Uses short cycles built around heavy prototyping and constant user involvement, aiming to get a usable system out fast. Best suited to smaller, time-boxed projects where the user can sit with the developers.

> ★ *Interview Tip: "Waterfall vs Agile" is one of the most common SE interview questions. The short answer: Waterfall assumes requirements are fixed and delivers once at the end; Agile assumes requirements will change and delivers in small increments so feedback can steer the project.*

Feasibility Study Types

- **Technical** – Can we build it with current technology and skills?
- **Economic** – Is it cost-effective? (Cost-Benefit Analysis)
- **Operational** – Will users actually accept and use the system?
- **Schedule** – Can it be delivered within the required timeframe?
- **Legal** – Does it comply with laws and regulations (e.g. data privacy)?

2. Requirements Engineering

Before a single line of code is written, someone has to figure out exactly what the software must do and how well it must do it. Getting requirements wrong is the single biggest source of expensive rework in software projects, which is why this topic comes up constantly in interviews.

Types of Requirements

Type	Description & Examples
Functional	What the system should do — Login, Search, Payment processing
Non-Functional	How the system behaves — Performance, Security, Scalability, Reliability
Domain	Requirements from the application domain — e.g., banking rules
User Requirements	High-level statements for end users (natural language)
System Requirements	Detailed technical specifications for developers

A quick way to remember the difference: functional requirements describe *features*; non-functional requirements describe *qualities* of those features. “Users can reset their password” is functional. “Password reset must complete in under 2 seconds for 95% of requests” is non-functional.

SRS Document (IEEE 830)

- **Introduction** – Purpose, scope, definitions, overview
- **Overall Description** – Product perspective, functions, user characteristics
- **Specific Requirements** – Functional, non-functional, external interface requirements
- **Appendices & Index**

■ *SRS qualities to remember (often asked as a list): Correct, Unambiguous, Complete, Consistent, Ranked (by importance), Verifiable, Modifiable, Traceable.*

Requirement Elicitation Techniques

- **Interviews** – One-on-one conversations with stakeholders
- **Questionnaires** – Surveys for large groups of users
- **Observation** – Watching users work in their real environment (ethnography)
- **Prototyping** – Build a rough model so users can react to something concrete
- **Use Case Analysis** – Identify actors and how they interact with the system
- **Brainstorming** – Group idea generation session
- **JAD Sessions** – Joint Application Development workshops with all stakeholders together

3. Software Design Principles

Good design isn't about clever code — it's about code that's easy to change later without breaking everything else. SOLID, coupling/cohesion, and design patterns are the vocabulary interviewers use to check whether you think this way.

SOLID Principles

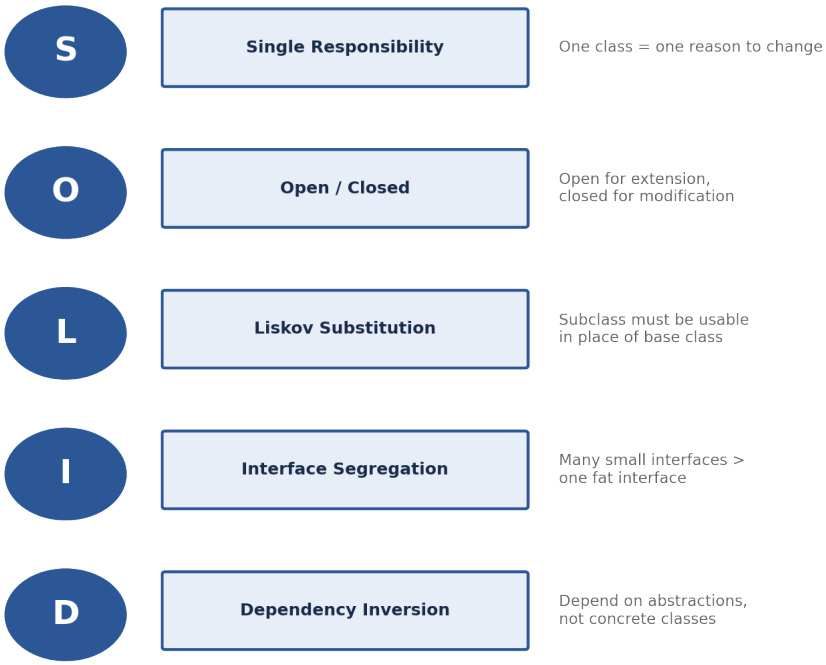

Figure 3.1: The five SOLID principles.

Principle	Meaning
S – Single Responsibility	A class should have only ONE reason to change
O – Open/Closed	Open for extension, CLOSED for modification
L – Liskov Substitution	Subtypes must be substitutable for their base types
I – Interface Segregation	Many specific interfaces > one general interface
D – Dependency Inversion	Depend on abstractions, NOT on concretions

Worked example (SRP): an **Invoice** class that calculates totals AND prints receipts AND saves to a database violates SRP — it has three reasons to change. Splitting it into **Invoice**, **InvoicePrinter**, and **InvoiceRepository** gives each class exactly one job and one reason to change.

Coupling & Cohesion

Coupling measures how dependent modules are on each other's internals — low coupling means you can change one module without breaking others. **Cohesion** measures how focused a single module is — high cohesion means everything inside a module genuinely belongs together. Good architecture aims for **low coupling, high cohesion**.

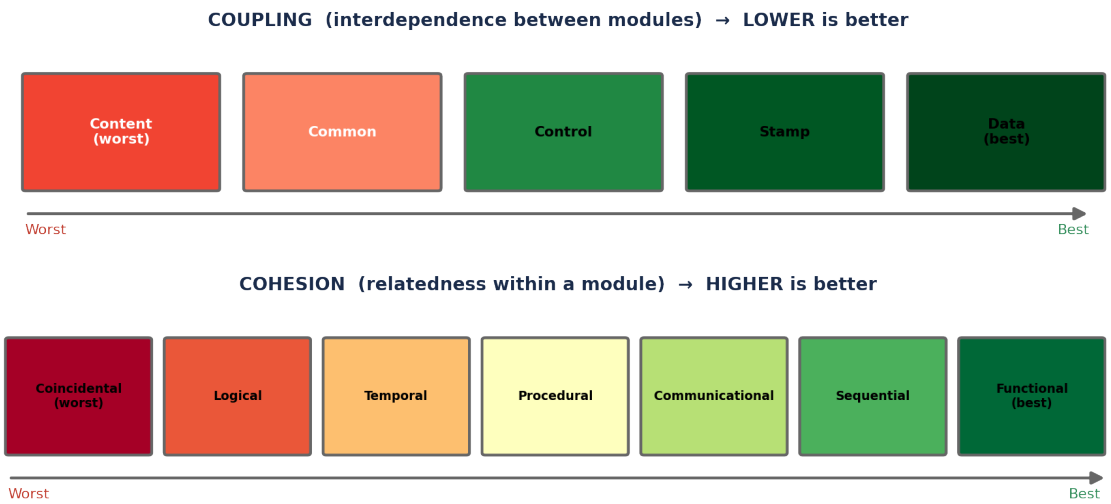

Figure 3.2: Coupling types (lower is better) and cohesion types (higher is better), ranked worst to best.

Coupling Type	Description
Content	One module modifies another's internal data (worst)
Common	Modules share global data
Control	One module controls the logic of another
Stamp	Modules share a composite data structure
Data	Modules share data only via parameters (best)

Design Patterns (Gang of Four — 23 Patterns)

A design pattern is a proven, reusable solution to a recurring design problem — not finished code, but a template you adapt to your situation. The 23 classic (“GoF”) patterns fall into three families based on what problem they solve.

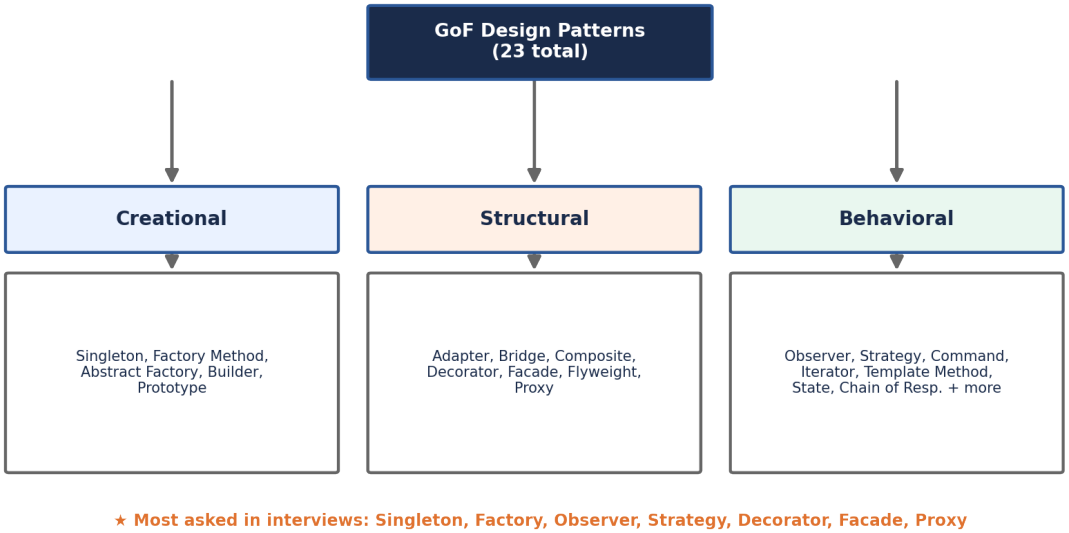

Figure 3.3: The three GoF pattern categories and their members.

Category	Patterns & One-liner
Creational (Object creation)	Singleton (one instance), Factory Method (subclass decides), Abstract Factory (family of objects), Builder (step-by-step), Prototype (clone)
Structural (Class composition)	Adapter (interface converter), Bridge (abstraction-impl split), Composite (tree), Decorator (add behavior), Facade (simplified interface), Flyweight (share objects), Proxy (surrogate)
Behavioral (Communication)	Observer (publish-subscribe), Strategy (interchangeable algorithms), Command (encapsulate request), Iterator, Template Method, State, Chain of Responsibility, Mediator, Memento, Visitor, Interpreter

★ Most asked in interviews: Singleton, Factory, Observer, Strategy, Decorator, Facade, Proxy — make sure you can explain each with a one-sentence real-world example (e.g., Observer = how a YouTube channel notifies all subscribers when it uploads a new video).

Architecture Patterns

- **MVC** – Model-View-Controller: separates data, UI, and logic
- **MVP** – Model-View-Presenter: Presenter handles all UI logic
- **MVVM** – Model-View-ViewModel: two-way data binding (Angular, WPF)
- **Layered (N-Tier)** – Presentation → Business Logic → Data Access
- **Microservices** – Small, independently deployable services
- **Event-Driven** – Components communicate via events/messages
- **SOA** – Service-Oriented Architecture; services communicate via an enterprise bus
- **Hexagonal (Ports & Adapters)** – Core business logic isolated from external systems

4. UML — Unified Modeling Language

UML is a standard set of diagram types for visualizing how a system is structured and how it behaves, so developers, testers, and stakeholders share one common picture instead of relying on scattered text descriptions.

Diagram Type	Purpose & Key Elements
Use Case	System functionality from the user's perspective. Actors, Use Cases, <<include>>, <<extend>>
Class Diagram	Static structure: classes, attributes, methods, relationships
Sequence	Object interactions over time; messages between lifelines
Activity	Workflow / flowchart; actions, decisions, forks, joins
State Machine	States an object goes through; events & transitions
Component	Physical components and their interfaces
Deployment	Hardware nodes and software deployment
Object Diagram	Snapshot of objects at a point in time
Package Diagram	Grouping of related classes into packages
Communication	Similar to sequence but emphasizes links between objects
Timing Diagram	Shows state changes over real time
Interaction Overview	Combines activity + sequence diagrams

Class Diagram Relationships

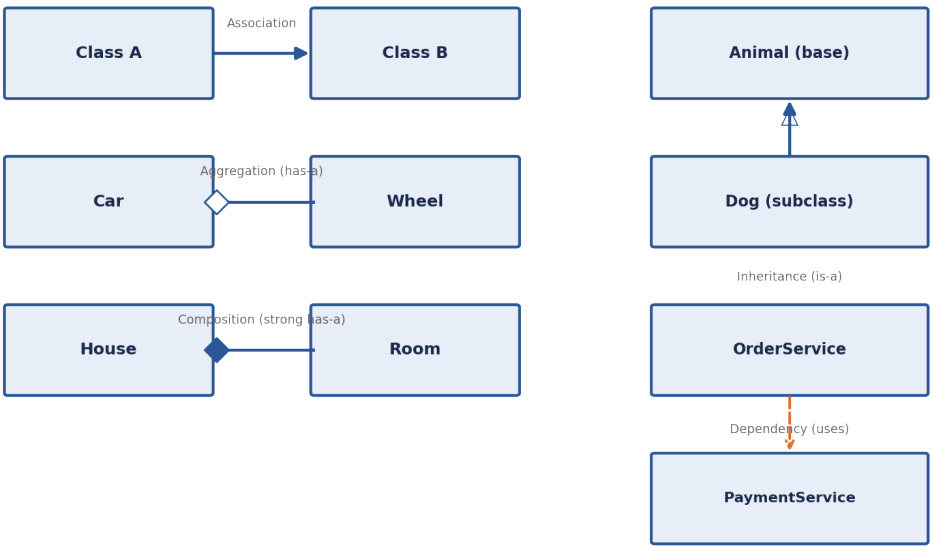

Figure 4.1: Association, aggregation, composition, inheritance, and dependency.

Relationship	Notation & Meaning
Association	Solid line – general relationship between classes
Aggregation	Hollow diamond – 'has-a' (child can exist independently)
Composition	Filled diamond – strong 'has-a' (child cannot exist alone)
Inheritance	Hollow triangle arrow – 'is-a' relationship
Realization	Dashed + hollow triangle – class implements an interface
Dependency	Dashed arrow – temporarily uses another class

Aggregation vs composition trips people up in interviews: think of a Car and its Wheels (aggregation — a wheel can be removed and reused elsewhere) versus a House and its Rooms (composition — a room stops existing if the house is demolished).

■ *Practice tip: be ready to sketch a Use Case + Class + Sequence diagram on a whiteboard for a common scenario like an ATM, Library System, or E-Commerce checkout — this is a frequent live-interview exercise.*

5. Agile Methodology & Scrum

Scrum is the most widely used Agile framework in industry, so almost every placement interview — whether for a developer, QA, or product role — expects you to know its vocabulary and rhythm.

Agile Manifesto — 4 Values

- Individuals and interactions OVER processes and tools
- Working software OVER comprehensive documentation
- Customer collaboration OVER contract negotiation
- Responding to change OVER following a plan

Scrum Framework

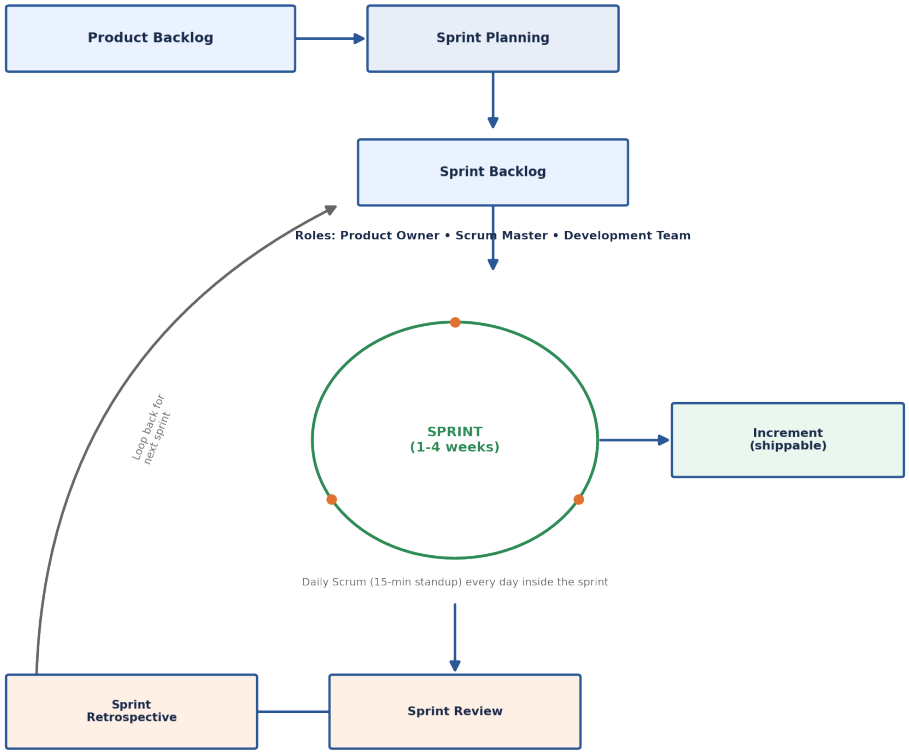

Figure 5.1: The Scrum cycle — backlog, sprint, daily scrum, review, retrospective.

Element	Details
Roles	Product Owner (PO), Scrum Master (SM), Development Team
Sprint	Time-boxed iteration of 1–4 weeks; produces a potentially shippable increment
Artifacts	Product Backlog (all features), Sprint Backlog (sprint tasks), Increment (done work)
Events	Sprint Planning, Daily Scrum (15 min standup), Sprint Review, Sprint Retrospective
DoD	Definition of Done: agreed criteria a story must meet to be 'done'
Velocity	Story points completed per sprint; used for forecasting

The Product Owner decides *what* gets built and in what order (represents the customer's interests); the Scrum Master protects the team's process and removes blockers (a coach, not a boss); the Development Team decides *how* to build it.

Kanban vs Scrum

Kanban	Scrum
Continuous flow	Fixed-length sprints
No roles defined	Defined roles (PO, SM, Team)
WIP limits	Sprint backlog commitment
Pull system	Push + Pull
Board visualization	Scrum board + ceremonies

Agile Estimation Techniques

- **Story Points** – Relative effort using Fibonacci numbers (1, 2, 3, 5, 8, 13…)
- **Planning Poker** – Team members simultaneously reveal cards, then discuss differences
- **T-Shirt Sizing** – XS, S, M, L, XL categories for quick estimates
- **Bucket System** – Items sorted into buckets of complexity

★ *User Story format: "As a [user], I want [feature] so that [benefit]." Acceptance criteria define exactly when that story counts as done.*

6. Software Testing

Testing questions in interviews check two things: do you understand the different levels at which bugs can be caught, and can you design test cases systematically instead of guessing inputs at random.

Levels of Testing

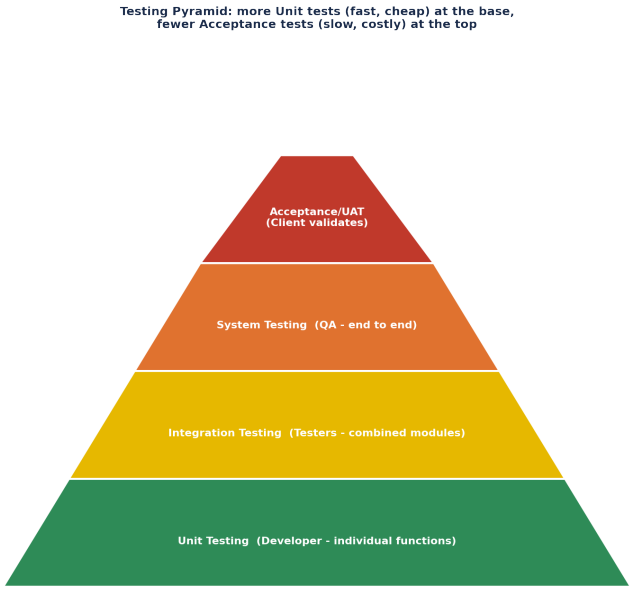

Figure 6.1: The testing pyramid — many fast unit tests at the base, fewer slow acceptance tests at the top.

Level	Done By / Purpose
Unit Testing	Developer; tests individual functions/methods in isolation
Integration Testing	Testers; tests interaction between combined modules
System Testing	QA team; end-to-end testing on the complete system
Acceptance Testing	Client/Users; UAT – validates against business requirements
Regression Testing	After changes; ensures existing features still work
Smoke Testing	Quick sanity check; is the build stable enough to test?
Sanity Testing	Narrow, focused check after a specific bug fix
Performance Testing	Load, Stress, Spike, Soak testing for non-functional requirements

V-Model Mapping

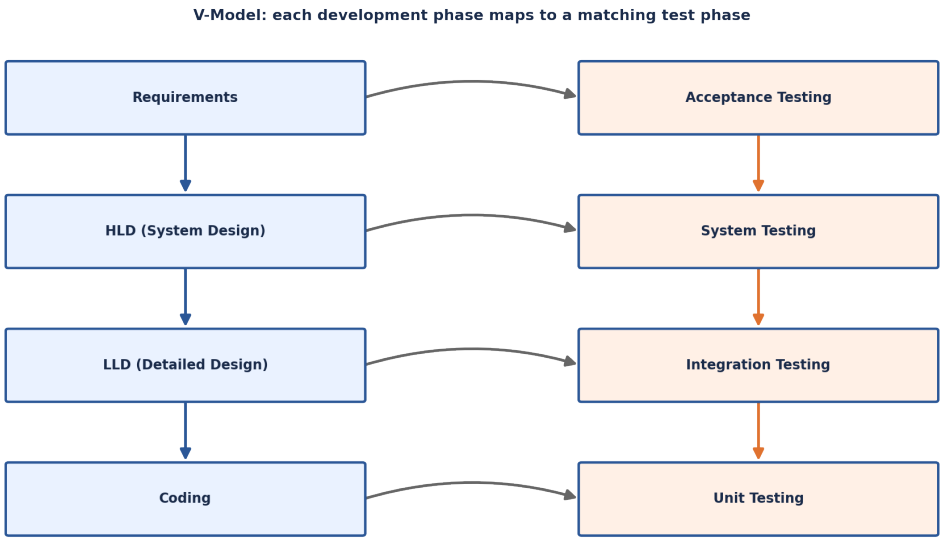

Figure 6.2: Each development phase has a corresponding test phase.

Black Box vs White Box vs Grey Box

Type	Tester Knows	Techniques
Black Box	Nothing about internals	Equivalence Partitioning, Boundary Value Analysis, Decision Table
White Box	Full source code access	Statement, Branch, Path Coverage
Grey Box	Partial knowledge	Combination of both; database testing

Equivalence Partitioning & Boundary Value Analysis

Equivalence Partitioning (EP): divide inputs into valid and invalid classes, then test just one representative value from each class instead of every possible input. Example: for age 18–60 as valid, test 30 (valid), 5 (invalid, too low), 70 (invalid, too high).

Boundary Value Analysis (BVA): most bugs hide at the edges, so test exactly at the boundaries — min, min+1, max-1, max, and just outside them. Example: for the range 1–100, test 0, 1, 2, 99, 100, 101.

Key Testing Terms

Term	Definition
Error/Mistake	Human action that produces an incorrect result
Defect/Bug	Flaw in software that causes incorrect behavior
Failure	System deviates from expected behavior at runtime
Test Case	Set of conditions + expected result for testing
Test Plan	Document describing scope, approach, resources, schedule
Test Suite	Collection of related test cases

Term	Definition
Alpha Testing	Testing by internal users at the developer's site
Beta Testing	Testing by external users in a real environment
Mutation Testing	Small changes (mutations) injected to verify test quality

★ *V-Model maps each dev phase to a test phase: Requirements → Acceptance Testing, Design → System Testing, Detailed Design → Integration Testing, Coding → Unit Testing.*

7. Software Metrics & Project Management

Estimation and metrics questions test whether you can talk about a project in terms a manager cares about: how big is it, how long will it take, and how do we know it's healthy.

Size & Effort Estimation

Technique	Description
LOC (Lines of Code)	Simple count; machine/language dependent
Function Points (FP)	Measures functionality: inputs, outputs, queries, files, interfaces
Use Case Points	Derived from use case complexity and technical factors
COCOMO	Constructive Cost Model: Effort = a × (KLOC)^b; Basic / Intermediate / Detailed
Planning Poker	Agile estimation using story points (team-based)
Delphi Technique	Expert opinion consensus through iterative rounds

COCOMO Model

Basic COCOMO estimates effort in person-months from the estimated size of the project in KLOC (thousand lines of code), using constants that reflect how complex the project environment is:

Mode	a	b	When to use
Organic	2.4	1.05	Small teams, familiar domain
Semi-detached	3.0	1.12	Mixed team experience
Embedded	3.6	1.20	Tight constraints (hardware, safety, etc.)

Key Software Metrics

Metric	Formula / Meaning
Cyclomatic Complexity	M = E – N + 2P (edges – nodes + 2 × components); measures code complexity
Defect Density	Defects / KLOC; lower = better quality
MTBF	Mean Time Between Failures = MTTF + MTTR
MTTF	Mean Time To Failure: average time system works before failure
MTTR	Mean Time To Repair: average time to fix a failure
Availability	MTTF / (MTTF + MTTR) × 100%
Test Coverage	% of code/branches/paths tested
Velocity	Story points completed per sprint (Agile)

Cyclomatic Complexity in particular comes up as a calculation question: draw the control-flow graph, count edges (E), nodes (N) and connected components (P, usually 1), then apply M = E – N + 2P. A value above 10 is a signal that the function should be refactored into smaller pieces.

Risk Management

- **Risk Identification** – List all potential risks

- **Risk Analysis** – Probability × Impact = Risk Exposure
- **Risk Planning** – Mitigation, avoidance, transfer, acceptance strategies
- **Risk Monitoring** – Continuous tracking throughout the project

Software Quality Attributes (ISO 25010)

- **Functionality** – Does it do what it should?
- **Reliability** – Does it perform consistently?
- **Usability** – Is it easy to use?
- **Efficiency/Performance** – Does it respond fast with minimal resources?
- **Maintainability** – Is it easy to modify and fix?
- **Portability** – Can it run on different environments?
- **Security** – Is it protected against unauthorized access?

8. SCM, Version Control & DevOps

Every engineering job — internship or full-time — expects working Git knowledge on day one. Interviewers often test this with quick scenario questions (“how would you undo a bad commit that's already pushed?”) rather than pure definitions.

Software Configuration Management (SCM)

- **Version Control** – Track changes to code (Git, SVN)
- **Change Management** – Process for requesting/approving changes
- **Build Management** – Automated build (Maven, Gradle, Make)
- **Release Management** – Planning and managing software releases
- **Configuration Audit** – Verify config items meet requirements

Git Branching & Merging

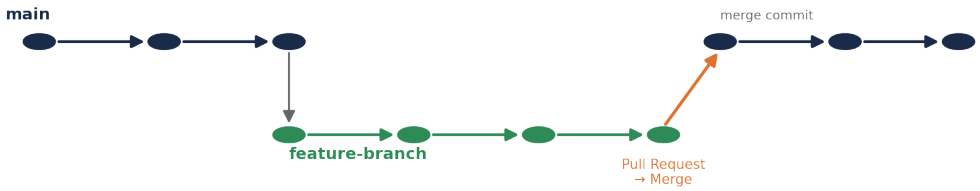

Figure 8.1: A feature branch is created from main, developed independently, then merged back via a pull request.

Concept	Description
Repository	Storage for code + full history
Commit	Snapshot of changes with a message
Branch	Independent line of development
Merge	Combine changes from two branches
Rebase	Move/combine commits onto another base; cleaner history
Pull Request	Request to merge feature branch → main (code review)
Conflict	Occurs when the same lines were changed in two branches
git stash	Temporarily save uncommitted changes
git cherry-pick	Apply a specific commit from another branch
git bisect	Binary search to find the commit that introduced a bug

CI/CD Pipeline

- **Continuous Integration (CI):** Automatically build & test code on every commit
- **Continuous Delivery (CD):** Automatically prepare a release to staging; manual deploy to production
- **Continuous Deployment:** Fully automated release straight to production
- **Tools:** Jenkins, GitHub Actions, GitLab CI, CircleCI, Travis CI

■ *Know the difference: Continuous **Delivery** still needs a human to click "deploy"; Continuous **Deployment** skips that click entirely and ships automatically.*

9. Software Maintenance, Reliability & Ethics

Software doesn't stop needing engineering work once it ships — in most companies, maintenance consumes more engineering time over a product's life than the original build did.

Types of Software Maintenance

Type	Description & Example
Corrective	Fix bugs discovered after release
Adaptive	Modify software to work in a new environment (OS upgrade, new hardware)
Perfective	Improve performance or add new features (user requests)
Preventive	Restructure/refactor code to prevent future problems (technical debt)

★ *Lehman's Laws: software must be continuously adapted, or it becomes progressively less satisfactory as its environment changes around it.*

Software Reliability

- **Reliability:** Probability that software performs its intended function correctly for a specified time
- **Fault Tolerance:** System continues to operate even when components fail
- **Redundancy:** Replication of critical components (hardware/software)
- **Failure Rate (λ)** = Number of failures / Total operating time
- **Reliability** R(t) = e^(–λt)

Software Re-engineering

- **Reverse Engineering:** Extract design from existing code
- **Re-engineering:** Restructure an existing system without changing its functionality
- **Forward Engineering:** Normal development from requirements to code
- **Refactoring:** Improve internal code structure without changing external behavior

Software Ethics (ACM/IEEE Code)

- **Public** – Act in the public interest
- **Client & Employer** – Act in the best interest of client and employer
- **Product** – Ensure software meets the highest professional standards
- **Judgment** – Maintain integrity and independence in professional judgment
- **Management** – Promote ethical management of software development

10. Top Interview Questions & Quick Answers

Q1. What is the difference between verification and validation?
Verification: Are we building the product RIGHT? (reviews, walkthroughs). Validation: Are we building the RIGHT product? (testing against actual requirements).

Q2. What is a software process model? Which one would you choose for a banking system?
A framework describing how software is developed. For banking (high reliability, stable requirements), the V-Model or Waterfall with rigorous testing phases is preferred.

Q3. Explain Cohesion and Coupling with an example.
Cohesion = how well a module's elements belong together (high is good). Coupling = dependency between modules (low is good). E.g., a PaymentProcessor class that only processes payments has high cohesion; it shouldn't depend on UI code, which keeps coupling low.

Q4. What is the difference between static and dynamic testing?
Static: testing without executing code (code reviews, walkthroughs, inspections). Dynamic: testing by executing the code (unit, integration, system testing).

Q5. What is a use case and how does it differ from a user story?
Use Case: a detailed description of system behavior with actors, preconditions, main flow, alternative flows. User Story: a short, informal description — 'As a [user], I want [goal] so that [reason]'. User stories are Agile-centric.

Q6. Explain the Singleton design pattern and where you'd use it.
Ensures only ONE instance of a class exists and provides a global access point. Common uses: database connections, Logger, Configuration manager, thread pool.

Q7. What is technical debt?
The implied cost of extra rework caused by choosing a quick, easy solution now instead of a better approach that would take longer. It accumulates over time and is managed through refactoring, code reviews, and good practices.

Q8. What is the difference between Alpha and Beta testing?
Alpha: done by the internal team at the developer's site before release. Beta: done by actual end users in a real environment before the final release.

Q9. What is Cyclomatic Complexity?
Measures the number of linearly independent paths through code. Formula: M = E – N + 2P. Lower is simpler; a value above 10 indicates high complexity.

Q10. What are SOLID principles? Give an example of SRP.
Five OOP design principles: Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion. SRP example: an 'Invoice' class should only handle invoice data, NOT printing — that belongs in a separate 'InvoicePrinter' class.

Q11. What is the difference between Agile and Scrum?
Agile is a philosophy/mindset built on 4 values and 12 principles. Scrum is a specific Agile FRAMEWORK with defined roles (PO, SM, Dev Team), events (Sprint, Standup), and artifacts.

Q12. What is Function Point Analysis?
A size-estimation technique based on functional complexity: counts External Inputs, Outputs, Inquiries, Internal Files, and External Interfaces. It's language-independent and useful for estimating effort and cost.

11. PLACEMENT ROADMAP — On-Campus vs Off-Campus

Knowing the theory is only half the job — you also need to know how the hiring process itself works so nothing catches you off guard. Both routes test similar fundamentals (DSA, CS core subjects, projects), but the sequence, pace, and pressure points differ. This section walks through both paths end to end.

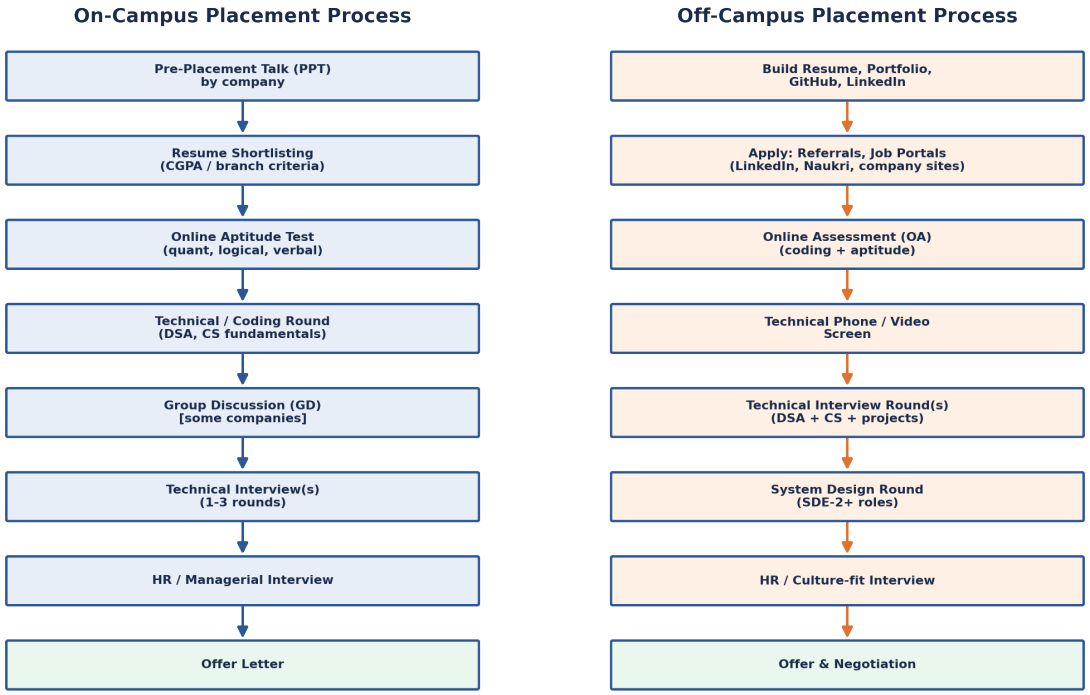

Figure 11.1: Typical on-campus vs off-campus hiring pipelines.

On-Campus Placements — What to Expect

On-campus drives are run by your college's placement cell, so the schedule, eligibility, and shortlisting rules are largely out of your hands — your job is to be ready when your turn comes.

- **Pre-Placement Talk (PPT):** the company presents the role, package, and eligibility criteria — attend even if you're unsure, since criteria and roles sometimes change after the talk.
- **Resume Shortlisting:** often filtered by CGPA cutoff, branch, and backlog history before anyone reads your resume closely — keep your CGPA and backlog record clean well before placement season starts.
- **Online Aptitude Test:** quantitative, logical reasoning, and verbal ability, usually with a strict time limit — speed matters as much as accuracy here.
- **Technical/Coding Round:** DSA problems on a platform like HackerRank or a custom portal, plus MCQs on CS fundamentals (OS, DBMS, CN, OOP).
- **Group Discussion:** used by some companies to test communication and teamwork — have a point of view but let others speak; dominating the conversation is scored negatively as often as staying silent.
- **Technical Interview(s):** live problem-solving, deep dives into your resume and projects, and CS fundamentals — expect 1–3 rounds depending on the company.
- **HR/Managerial Round:** culture fit, salary expectations, and questions like “why this company” — answers should be honest and specific, not generic.

★ On-campus tip: many colleges have a “one offer freeze” or “dream company” policy that restricts which companies you can sit for once you have an offer. Learn your college's exact rules early so you don't accidentally lock yourself out of a company you wanted.

Off-Campus Placements — What to Expect

Off-campus hiring (applying directly to companies, not through your college) has no fixed calendar — you drive the process yourself, which means more effort up front but no dependency on your college's placement cell or CGPA cutoffs.

- **Build your presence first:** a clean resume, an active GitHub with a few solid projects, and a complete LinkedIn profile are the baseline — recruiters and referrers check all three.
- **Apply broadly:** job portals (LinkedIn, Naukri, Wellfound), company career pages, and — most effective of all — referrals from alumni or LinkedIn connections at the company.
- **Online Assessment (OA):** usually a mix of DSA coding questions and aptitude, similar to on-campus tests but sometimes harder since off-campus roles compete against a national or global applicant pool.
- **Technical Phone/Video Screen:** a shorter, earlier filter than on-campus processes typically have — be ready to talk through your resume and solve a problem live over a call.
- **Technical Interview Round(s):** deeper DSA, CS fundamentals, and detailed questions about your projects — expect "why did you choose this approach" follow-ups more than in on-campus rounds.
- **System Design Round:** common for off-campus SDE roles beyond entry level, even sometimes for strong fresher candidates — practice basic scalability concepts (load balancing, caching, database choice) even as a student.
- **HR/Culture-fit Interview:** similar to on-campus, but you should research the company more independently since there's no placement cell briefing to rely on.
- **Offer & Negotiation:** off-campus offers are more often negotiable than on-campus fixed packages — it's reasonable to ask about compensation once an offer is extended.

> ★ *Off-campus tip: referrals dramatically increase your odds of getting past the initial resume screen, since many companies prioritize referred applications. Reaching out politely to alumni or engineers at your target company on LinkedIn is one of the highest-leverage things you can do.*

On-Campus vs Off-Campus — Side by Side

Aspect	On-Campus	Off-Campus
Who controls timing	College placement cell	You
Eligibility filter	CGPA / branch / backlog rules	Usually none, but competition is wider
Initial screen	Resume shortlist by cell + company	Resume/referral into an applicant tracking system
Competition pool	Your batch/college only	National or global applicant pool
Number of rounds	Often fewer (streamlined process)	Can be more, including system design
Salary negotiation	Usually fixed package	Often negotiable
Best lever	Strong academics + fundamentals	Strong projects + referrals + portfolio

Preparation Priorities Common to Both Routes

- **Data Structures & Algorithms:** arrays, strings, linked lists, trees, graphs, dynamic programming — this is the single highest-weight topic across both routes.
- **CS Core Subjects:** Operating Systems, DBMS, Computer Networks, and this very subject — Software Engineering — all show up in MCQ rounds and interview questions.
- **Projects:** 1–2 solid, well-explained projects beat five shallow ones — be ready to justify every technical decision you made.

- **Resume:** one page, quantified achievements (“reduced load time by 40%” beats “improved performance”), no unexplained buzzwords.
- **Mock Interviews:** practicing out loud, even with a friend, closes the gap between knowing an answer and explaining it clearly under pressure.

■ *Final tip: whichever route you're on, revise this document's SDLC models, SOLID principles, and design patterns sections a day before any interview — they are the most frequently recycled Software Engineering questions across both on-campus and off-campus interviews.*

12. Quick Revision Cheat Sheet

Key Formulas

Formula	Description
M = E – N + 2P	Cyclomatic Complexity
Effort = a(KLOC)^b	COCOMO Basic
MTBF = MTTF + MTTR	Mean Time Between Failures
Availability = MTTF/MTBF	System Availability (×100%)
R(t) = e^(–λt)	Reliability function
Defect Density = D/KLOC	Quality metric
FP = UFP × CAF	Function Points (CAF = complexity adjustment)
Risk Exposure = P × I	Probability × Impact

Acronyms You Must Know

Acronym	Full Form
SRS	Software Requirements Specification
SDD	Software Design Document
HLD / LLD	High-Level Design / Low-Level Design
SDLC	Software Development Life Cycle
SCM	Software Configuration Management
UAT	User Acceptance Testing
TDD / BDD	Test-Driven Development / Behavior-Driven Development
DDD	Domain-Driven Design
CI/CD	Continuous Integration / Continuous Delivery
KLOC	Kilo Lines of Code (1000 LOC)
FP	Function Points
MTBF	Mean Time Between Failures
COCOMO	Constructive Cost Model

Last-Minute Power Tips

- Waterfall: requirements fixed. Agile: requirements can change. Spiral: high-risk projects.
- Coupling: lower is better. Cohesion: higher is better. Remember this always!
- V-Model = Waterfall + testing planned at each phase. Best for safety-critical systems.
- Singleton, Factory, Observer, Strategy are the 4 most frequently asked design patterns.
- Unit test → Integration test → System test → Acceptance test (bottom-up pyramid).
- Verification = process quality; Validation = product quality.

- Cyclomatic Complexity > 10 = code needs refactoring. M = E – N + 2P.
- On-campus = college-controlled timeline & fixed package. Off-campus = self-driven, referral-powered, negotiable.

★ Best of luck for your placements! Revise SOLID principles, SDLC models, design patterns, and the placement roadmap daily in the final week. ✓